Campus Service Management System

Student Campus Operations Platform

Technical Project Report

Comprehensive System Documentation

Architecture | Design Patterns | OOP Concepts | SOLID Principles

Research and Development Personnel	
Samarth Chaudhary	Team Member
Aabir Sarkar	Team Member
Kushagra Maheshwari	Team Member
Anugra	Team Member
Vachan Gupta	Team Member

Rishihood University

1. Executive Summary

Campus Service Management System (CSMS) is a full-stack campus operations platform built to modernize and digitize service request management and facility operations within a university setting. The system bridges the gap between resident students and campus administrators, delivering a transparent, real-time, and efficient digital experience for all stakeholders.

1.1 Platform Overview

The platform serves two primary user roles with tailored dashboards and workflows:

- **Residents (Students)** can raise and track service requests, view available campus facilities, receive notifications, and monitor the status of their submitted tickets from a dedicated resident dashboard.
- **Administrators** have system-wide oversight including user management, service request board management, facility operations, analytics dashboards, and full booking visibility.

Mission Statement

CSMS's mission is to eliminate the inefficiency and opacity of traditional campus service operations by delivering a secure, real-time, and feature-rich digital platform that empowers students to request services seamlessly and enables administrators to manage campus operations with professional-grade tools.

1.2 Core Value Proposition

- **Typed REST API** — All endpoints are written in TypeScript with strict type safety, preventing runtime errors.
- **Role-based workflows** — Resident and admin dashboards are cleanly separated with appropriate data visibility per role.
- **Session-backed authentication** — Secure login with credentials derived from student enrollment numbers.
- **Fail-fast validation** — All environment variables and request payloads are validated at startup and the network boundary respectively.
- **Seed-driven development** — A comprehensive seed script provisions students, facilities, requests, bookings, and notifications for immediate dashboard coverage.

2. Technology Stack

CSMS employs a modern, production-grade technology stack designed for scalability, type safety, and developer productivity. Every technology choice was deliberate, balancing performance, ecosystem maturity, and long-term maintainability.

Category	Technology	Purpose
Backend Runtime	Node.js + Express 5	High-performance async HTTP server
Language	TypeScript	End-to-end type safety across backend and frontend
ORM	Prisma ORM	Type-safe database client with migration support
Database	PostgreSQL (Neon)	Relational database with ACID compliance
Frontend Framework	Next.js App Router	Server-side rendering & React-based UI
Frontend Language	TypeScript + React	Type-safe component-driven interface
Security	Helmet + CORS	HTTP hardening and cross-origin control
Auth	Session + bcryptjs	Secure password hashing and session login flow
Config Validation	dotenv + custom guard	Fail-fast environment variable validation

2.1 Backend Stack Deep Dive

The backend is built on Node.js with Express 5, leveraging TypeScript for complete type coverage. Prisma ORM abstracts raw SQL into a type-safe query interface, while PostgreSQL (hosted on Neon) serves as the persistence layer with full ACID transactional support.

- Express 5 introduces improved async error handling and enhanced router-level middleware that simplify route organization.
- Prisma generates a fully typed client from the schema, eliminating runtime type mismatches between application and database layers.
- Helmet adds essential HTTP security headers, while CORS is configured for controlled cross-origin requests from the Next.js frontend.
- bcryptjs provides secure password hashing; plaintext credentials are never stored.
- Environment variables are validated at startup — missing required values fail fast before the server accepts any requests.

2.2 Frontend Stack Deep Dive

The frontend is a Next.js application using the App Router paradigm, written in TypeScript with React. It consumes all backend REST endpoints and authenticates through a session-backed login flow proxied via Next.js API routes.

- Next.js App Router enables server components and file-based routing, reducing client bundle sizes.
- TypeScript ensures type-safe API consumption — response shapes are typed to match backend DTOs.

• Request creation is validated server-side and proxied through a Next.js API route, keeping backend credentials off the client.

• The frontend fetches all dashboard data — resident or admin — from dedicated typed backend endpoints.

3. System Architecture & Design

CSMS follows a modular, layered architecture that cleanly separates concerns across feature domains. The system is designed around the principle of high cohesion within modules and low coupling between them.

3.1 Backend Architecture

The backend is organized into feature modules, each encapsulating all related logic for a specific domain. This modular structure ensures that adding or modifying one feature domain does not impact unrelated modules.

Module	Responsibilities
Auth	User login, session management, password validation, credential hashing
Dashboard	Resident and admin dashboard data aggregation endpoints
Requests	Service request CRUD, board view, ticket form configuration
Facilities	Facility listing, availability queries, booking records
Analytics	Operational metrics aggregation for admin analytics view
Header/Profile	User profile data for navigation header (GET /api/header/me)
Health	Liveness probe endpoint (GET /api/health)

3.1.1 Layered Architecture Pattern

Each backend module follows a strict three-layer pattern:

- **Controller Layer** — Handles HTTP request parsing, response formatting, and route-level validation. Controllers are thin and delegate all business logic to services.
- **Service Layer** — Contains all business logic, orchestrates database operations, enforces rules, and manages cross-module interactions.
- **Data Layer (Prisma)** — Handles all database interactions. Services interact with Prisma models; no raw SQL is used in application code.

3.2 Frontend Architecture

The frontend follows a component-based architecture with distinct organizational layers that separate UI rendering from data fetching and business logic.

- **Pages** — Top-level App Router segments that compose smaller components and fetch data from backend endpoints.
- **Components** — Reusable, stateless or locally stateful UI elements with clear prop interfaces.
- **API Routes** — Next.js API routes proxy sensitive backend calls (e.g., POST /api/requests) keeping credentials and validation server-side.
- **Services** — Typed fetch wrappers that abstract all HTTP communication, providing typed request and response functions.

3.3 Database Design

The database schema is designed in PostgreSQL with Prisma, structured around core entities that reflect the real-world relationships within campus operations.

Entity	Key Fields
User	id, enrollmentNumber (unique), passwordHash, role, createdAt, updatedAt
ServiceRequest	id, title, description, status, priority, category, userId (FK), createdAt
Facility	id, name, type, location, capacity, status, description
Booking	id, facilityId (FK), userId (FK), startTime, endTime, status, createdAt
Notification	id, userId (FK), message, type, read, createdAt

3.3.1 Key Schema Decisions

• **Enrollment-based auth** — Users are identified by enrollment number, enabling bulk provisioning from a CSV seed file.

• **Role field on User** — A single role enum (RESIDENT / ADMIN) drives dashboard routing and endpoint access control.

• **ServiceRequest status lifecycle** — Requests move through PENDING → IN_PROGRESS → RESOLVED states, providing a clear audit trail.

• **Facility bookings** — The Booking entity links a User to a Facility with time-range fields, supporting overlap detection.

• **Notifications** — Per-user notification records with a read flag enable lightweight notification management without a message queue.

4. API Endpoints

The backend exposes a typed REST API consumed entirely by the Next.js frontend. All endpoints return JSON. Protected endpoints validate the session before serving data.

Method	Endpoint	Description
GET	/api/health	Liveness probe — returns server status
POST	/api/auth/login	Authenticate with enrollment number and password
GET	/api/header/me?userId=...	Fetch user profile for navigation header
GET	/api/dashboard/resident?userId=...	Resident dashboard data (requests, notifications, bookings)
GET	/api/dashboard/admin	Admin dashboard summary (all requests, analytics preview)
GET	/api/analytics	Operational analytics metrics for admin view
GET	/api/facilities	List all campus facilities with status
GET	/api/requests/board?userId=...	Kanban board of service requests for a user
GET	/api/ticket/form-config	Dynamic form configuration for request creation
POST	/api/requests	Create a new service request (server-validated)

5. Object-Oriented Programming Concepts

CSMS is architected with a deliberate application of OOP principles, ensuring that real-world domain concepts (service requests, facilities, users, notifications) are modeled as coherent, reusable, and maintainable software entities.

5.1 Encapsulation

Business logic and data are encapsulated within well-defined service classes, preventing unauthorized direct database access and ensuring that implementation details do not leak across module boundaries.

- **RequestService** — Encapsulates all service request lifecycle logic: creation, status transitions, board aggregation, and ownership validation.
- **AuthService** — Contains credential verification, password hashing comparison, and session token management. External consumers call authenticate() without knowing the internal bcrypt comparison chain.
- **FacilityService** — Encapsulates facility queries and booking overlap detection. Consumers call getAvailable() without knowing the SQL-level time range logic.
- **AnalyticsService** — All metrics aggregation is hidden behind a single getMetrics() call; no raw Prisma aggregations are exposed to the controller.

5.2 Abstraction

Complexity is hidden behind clean, simple interfaces. High-level components interact with abstractions rather than implementations, enabling internal complexity to grow without affecting consumers.

The DashboardController calls dashboardService.getResidentData(userId) and receives a fully typed aggregate object. It has no awareness of how many Prisma queries were executed internally or how data was joined and shaped. This abstraction boundary means the data aggregation strategy can evolve without touching any controller.

5.3 Inheritance & Polymorphism

TypeScript interfaces define contracts that multiple implementations satisfy interchangeably. Service interfaces allow controllers to depend on abstractions rather than concrete classes, supporting polymorphic substitution and testability.

- IRequestService and its concrete RequestService implementation can be swapped with a mock in unit tests without changing any controller code.
- Shared base response shapes (ApiResponse) ensure all controllers return polymorphically compatible typed structures.
- Role-based middleware applies polymorphically to any route — the same guard function handles both RESIDENT and ADMIN role checks through a single configurable parameter.

5.4 Composition Over Inheritance

CSMS preferentially uses composition to build complex behaviors from simple, reusable parts rather than deep inheritance hierarchies that create tight coupling.

• DashboardService composes RequestService, FacilityService, and NotificationService to aggregate dashboard data rather than inheriting from a base service.

• Middleware is composed on routes — authGuard, roleGuard, and validate middleware are chained as independent, composable units.

• The seed script composes individual entity factories (createUser, createFacility, createRequest) to build a complete database state.

6. SOLID Design Principles

The SOLID principles are systematically applied throughout CSMS, ensuring the system remains maintainable, extensible, and testable as it grows in complexity.

Letter	Principle	Core Idea
S	Single Responsibility	Every class has exactly one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes are substitutable for base types
I	Interface Segregation	Clients depend only on methods they use
D	Dependency Inversion	Depend on abstractions, not concretions

6.1 S — Single Responsibility Principle (SRP)

Every class in CSMS has a single, well-defined responsibility.

- Controllers — Exclusively handle HTTP request/response lifecycle. No business logic lives in controllers.
- Services — Exclusively handle business rules and data orchestration. No HTTP-specific code exists in services.
- Middleware — Each middleware function (authGuard, roleGuard, validate) has a single cross-cutting concern.
- Seed functions — Each seeding function (seedStudents, seedFacilities, seedRequests) handles exactly one entity type.

6.2 O — Open/Closed Principle (OCP)

The system is open for extension but closed for modification. New capabilities can be added without altering existing, tested code.

- New service request categories are supported by adding entries to the form-config response — no controller logic changes.
- New dashboard widgets are added as new service methods composed into DashboardService — existing methods are untouched.
- New API endpoints are new route files and controllers — the existing routing infrastructure is extended, not modified.
- New analytics metrics are new aggregation methods in AnalyticsService — the controller interface remains identical.

6.3 L — Liskov Substitution Principle (LSP)

All concrete service implementations can be substituted through their interface contracts. Controllers never need to check which concrete implementation they hold.

• A mock RequestService satisfying IRequestService can replace the real service in tests — all controller behavior remains valid.

• Any class implementing IAuthService can be injected into AuthController — the substitution is seamless.

• Middleware functions satisfy a common Express RequestHandler signature, allowing any middleware to be substituted or composed without breaking the chain.

6.4 I — Interface Segregation Principle (ISP)

CSMS uses specific, narrow interfaces and Data Transfer Objects (DTOs) so that consuming classes depend only on the subset they actually need.

• CreateRequestDTO contains only the fields required for request creation — no unnecessary fields from the full ServiceRequest model.

• ResidentDashboardDTO exposes only resident-relevant data — admin-specific fields are absent from the resident interface.

• Service interfaces expose only the methods relevant to their consumers, avoiding god-object interfaces.

6.5 D — Dependency Inversion Principle (DIP)

High-level modules (Controllers) depend on abstractions (Service interfaces), not on concrete database implementations.

• RequestController depends on IRequestService, not on RequestService or Prisma directly.

• RequestService depends on the Prisma abstraction layer, not on raw PostgreSQL queries.

• This chain of abstraction means unit tests can inject mock services into controllers without spinning up a database.

7. Design Patterns

CSMS implements established Gang of Four design patterns, each chosen to solve a specific architectural challenge in a clean, idiomatic TypeScript way.

7.1 Singleton Pattern — Database Client

Location: src/config/database.ts

The Prisma client is instantiated once as a module-level singleton. All service classes import and share this single instance, ensuring connection pool efficiency and preventing multiple database connections from being opened unnecessarily.

- Prisma maintains an internal connection pool. Creating multiple PrismaClient instances would open multiple pools, exhausting Neon database connections.
- The Singleton Pattern guarantees exactly one pool exists for the application's lifetime.
- All services import the shared prisma instance — no service creates its own client.

7.2 Factory Pattern — Seed Data Generation

Location: prisma/seed.js

The seed script uses factory functions to create entity instances with consistent, production-shaped data. Each factory encapsulates the construction logic for a specific entity type, enabling bulk creation without duplicating instantiation logic.

- createStudentUser(enrollment) — Factory that constructs a User record with hashed credentials derived from the enrollment number.
- createFacility(data) — Factory that constructs a Facility with validated fields and default status.
- createServiceRequest(userId, data) — Factory that constructs a ServiceRequest with appropriate status and priority defaults.
- Factories are composable — the seed script orchestrates them in dependency order without coupling their internals.

7.3 Strategy Pattern — Role-Based Dashboard

Location: src/modules/dashboard/

The dashboard module applies the Strategy Pattern implicitly: based on the authenticated user's role, a different data aggregation strategy is executed. The controller selects the strategy (resident or admin) and delegates to it uniformly.

- ResidentDashboardStrategy — Fetches and shapes data relevant to a single student: their requests, bookings, and notifications.
- AdminDashboardStrategy — Aggregates system-wide data: all pending requests, occupancy rates, recent activity.
- The controller calls getResidentData() or getAdminData() uniformly — the strategy's internal complexity is hidden.

• Adding a new role (e.g., Maintenance Staff) requires only a new strategy class — the controller composition logic is unchanged.

7.4 Observer Pattern — Notification System

Location: src/modules/notifications/

When key domain events occur (service request status change, booking confirmation), the system creates Notification records for affected users. Consumers (the frontend dashboard) observe the notification state via polling the notifications endpoint.

• Status change in RequestService triggers NotificationService.createForUser() — the request module does not directly manipulate notification records.

• This decoupled event → notification chain means new notification triggers can be added without modifying existing service logic.

• The frontend dashboard's notification polling reflects real-time state changes without requiring WebSocket infrastructure.

7.5 Template Method Pattern — Middleware Pipeline

Location: src/middleware/

Express middleware chains define a template for request processing: parse → authenticate → authorize → validate → handle → respond. Each middleware step is a hook that can be overridden or omitted per route, while the overall processing skeleton remains constant.

• authGuard always runs before roleGuard — the sequence template is enforced by route registration order.

• Validation middleware (validate(schema)) is parameterized, making it a reusable template hook for any Zod schema.

• Error handling middleware at the end of the chain acts as the template's catch-all step, uniformly formatting all errors.

8. Key Features & Modules

8.1 Service Request Management

The service request module is the operational core of CSMS, providing a complete ticket lifecycle from creation to resolution.

- Dynamic form configuration — The /api/ticket/form-config endpoint serves form field definitions dynamically, allowing form shapes to change without frontend deployments.
- Kanban board view — /api/requests/board returns requests grouped by status (PENDING, IN_PROGRESS, RESOLVED) for a drag-and-drop-ready board view.
- Priority management — Requests carry Low / Medium / High priority, enabling admins to triage incoming tickets.
- Category classification — Requests are classified by category (Maintenance, Cleaning, IT, Security, etc.) for routing and analytics.
- User ownership — Each request is linked to the creating user, enabling resident-scoped views and accountability tracking.

8.2 Facility Management & Booking

The facilities module provides a structured view of all campus facilities and manages booking records.

- Facility listing with status — All facilities are served with current status (Available / Occupied / Under Maintenance).
- Booking records — The Booking entity captures who booked which facility, for what time range, and current booking status.
- Capacity field — Facilities carry a capacity field enabling room-size-aware search filtering.
- Booking visibility on dashboards — Resident dashboards surface the user's upcoming bookings; admin dashboards show system-wide booking activity.

8.3 Analytics Dashboard

The analytics module aggregates operational data for administrators to understand campus service health.

- Request volume metrics — Total, pending, in-progress, and resolved request counts for at-a-glance operational status.
- Facility utilization — Booking counts per facility enable occupancy insight.
- Response time indicators — Time-from-creation-to-resolution metrics surface operational efficiency trends.
- Category breakdown — Request distribution across categories helps prioritize staffing and resource allocation.

8.4 Role-Based Access Control (RBAC)

CSMS enforces strict role separation through session-decoded role claims verified by server-side middleware.

• Residents — Access to personal service request creation and tracking, facility browsing, and personal bookings and notifications.

• Administrators — System-wide access to all service requests, full facility management, analytics, and user management.

• Route guards — roleGuard middleware is applied per route, ensuring residents cannot access admin endpoints and vice versa.

• Dashboard segregation — Separate API endpoints (/dashboard/resident vs /dashboard/admin) enforce data boundary at the API level.

8.5 Notification System

A lightweight notification system keeps residents informed of status changes without requiring real-time infrastructure.

• Per-user notification records with read/unread state enable a notification bell UI pattern.

• Notifications are triggered by domain events: request status changes, booking confirmations.

• Resident dashboard surfaces unread notification count and recent notification list.

• Mark-as-read functionality clears notification state without deleting records, preserving audit history.

9. Security & Performance

9.1 Security Architecture

• **bcryptjs Password Hashing** — User passwords are hashed with bcrypt before storage; plaintext credentials are never persisted in the database.

• **Helmet HTTP Headers** — Helmet middleware sets Content-Security-Policy, X-Frame-Options, X-XSS-Protection, and other hardening headers on all responses.

• **CORS Configuration** — Only the configured FRONTEND_ORIGIN is allowed, preventing unauthorized cross-origin access.

• **Environment Variable Validation** — Required configuration (DATABASE_URL, PORT, FRONTEND_ORIGIN) is validated at startup; the server refuses to start with missing values.

• **Role Guards** — Middleware verifies role claims on every protected route, ensuring residents cannot access admin endpoints.

• **Server-side Request Validation** — POST /api/requests is validated server-side through the Next.js proxy, preventing malformed data from reaching backend business logic.

9.2 Performance Optimizations

• **Prisma Query Optimization** — Only required fields are selected in database queries using Prisma select, avoiding over-fetching of large text or JSON fields.

• **Singleton Database Client** — A single Prisma connection pool serves all requests, maximizing PostgreSQL connection efficiency on Neon.

• **Stateless API Design** — The REST API is stateless; each request carries all necessary context via session token. This enables horizontal scaling without session affinity.

• **Next.js Static Optimization** — Pages that do not require per-request data are statically generated at build time, reducing server load.

• **Fail-Fast Validation** — Invalid requests are rejected at the network boundary before reaching database operations, saving unnecessary query costs.

10. System Design Principles

Beyond SOLID and OOP, CSMS adheres to broader system design principles that ensure long-term reliability, observability, and maintainability.

10.1 Separation of Concerns (SoC)

Every layer of the system has a distinct, non-overlapping responsibility. The HTTP layer knows nothing about analytics aggregation. The analytics layer knows nothing about session management. The database layer knows nothing about business rules.

10.2 DRY — Don't Repeat Yourself

Common logic is extracted into shared utilities and middleware. Authentication checks, role validation, and error formatting are each defined once and reused across all modules.

10.3 Fail Fast & Validate Early

Environment variable validation runs at startup. Request payload validation runs at the middleware layer before business logic executes. Invalid states are surfaced at the earliest possible point.

10.4 Stateless API Design

The REST API is stateless — each request contains all information needed to process it. No server-side session state is maintained in memory, enabling horizontal scaling of the backend without session affinity requirements.

10.5 Seed-Driven Development

The comprehensive seed script allows any developer to spin up a fully populated, production-shaped environment in seconds. This principle reduces "works on my machine" issues and ensures consistent data for frontend development and testing.

11. Development Status & Future Roadmap

11.1 Current Progress

CSMS has completed its foundational milestone with the following capabilities fully operational:

- PostgreSQL database schema complete with Prisma migration and all core entity models.
- REST API with 10 typed endpoints serving resident and admin workflows.
- Session-backed authentication with enrollment-number-based login.
- Role-based middleware with resident and admin route separation.
- Service request lifecycle (PENDING → IN_PROGRESS → RESOLVED) operational.
- Facility listing and booking record management active.
- Analytics endpoint aggregating operational metrics for admin dashboard.
- Notification system with per-user records and read/unread state.
- Seed script provisioning students, facilities, requests, bookings, and notifications.
- Next.js frontend consuming all backend endpoints with typed API client.

11.2 Future Roadmap

Feature	Description
Real-Time Notifications	WebSocket integration for push notifications on request status changes, eliminating polling
Mobile Application	React Native port of the resident-facing interface for iOS and Android platforms.
Payment Integration	Stripe or Razorpay integration for facility booking fee collection and receipts.
AI Request Routing	ML-based categorization and priority assignment for incoming service requests.
Advanced Analytics	Occupancy heatmaps, response-time trend analysis, and seasonal staffing insights.
Multi-Campus Support	Multi-tenancy architecture to serve multiple campus locations from a single deployment.
Email Notifications	SMTP integration for booking confirmations, request updates, and admin alerts.
Document Uploads	File attachment support for service requests (photos of maintenance issues, etc.).

12. Conclusion

The Campus Service Management System represents a well-architected, full-stack solution to the real-world challenge of campus operations management. Its technical depth is evidenced by the deliberate application of OOP principles, SOLID design guidelines, and proven Gang of Four design patterns across a modern TypeScript codebase.

The Singleton Pattern ensures resource-efficient database access. The Factory Pattern enables consistent, composable seed data generation. The Strategy Pattern cleanly separates resident and admin data concerns. The Observer Pattern provides a decoupled notification mechanism. The Template Method Pattern enforces a consistent, extensible middleware processing pipeline.

Together, these patterns form a codebase that is not only functional today but designed to scale and evolve with minimal technical debt. CSMS achieves a rare combination of pedagogical clarity and production readiness — demonstrating OOP, SOLID, and system design principles in concrete, working implementations rather than abstract theory.

Technical Excellence Summary	
OOP Principles	Encapsulation, Abstraction, Polymorphism, Composition
SOLID Principles	All five principles applied throughout the codebase
Design Patterns	Singleton, Factory, Strategy, Observer, Template Method
Technology Stack	TypeScript, Node.js, Express 5, Prisma, PostgreSQL, Next.js
API Coverage	10 typed REST endpoints covering all domain operations
Security	bcrypt, Helmet, CORS, role guards, fail-fast validation